

Rapport d'Intégration

Santana v2.0 – Améliorations architecturales et
fonctionnelles

Date : 24 juin 2026

Agent : Hermès (Black Intelligence)

Contexte : Conseil de Discipline BTR – 4 agents

De : Santana 22 outils, 1 provider, skills SQLite

À : Santana 27 outils, 2 providers, skills fichiers .md

22 correctifs • 142 tests verts • 0 régression

1. Résumé exécutif

Cette session a consisté à **copier ce qui marche** — principe inspiré de la vidéo « Comment devenir riche si vite qu'on a l'impression de TRICHER » (Un Peu Mieux, 20 juin 2026). Au lieu de réinventer, Santana a copié les fonctionnalités et patterns architecturaux éprouvés des autres agents BTR : Hermès Agent, Claude Code et OpenClaw.

22

Correctifs

142

Tests verts

27

Outils

17k

Lignes Python

Conseil de Discipline

4 agents BTR ont analysé la codebase de Santana et identifié 40 propositions (10×4) de ce qu'elle devrait copier. Convergence sur :

- **P0 (4/4 accord)** : Multi-provider, Skills .md, fs_write, delegate_task, Validation sorties
- **P1 (2-3/4 accord)** : Healthcheck, Zéro rouge, Profil injecté, Checklist, SOUL.md
- **P2 (1-2/4 accord)** : Asymétrique, Logs JSON, Cache SHA256, Token counting, Cron, Nexus

2. Architecture : Avant / Après

Architecture précédente

```
Santana (monolithe Python)
└─ santana.py (272 lignes, tout)
    └─ react_loop.py (670 lignes, while séquentiel)
    └─ deepseek_client.py (267 l., 1 provider)
    └─ tools/tools.py (664 l., 22 outils)
    └─ memory.py (126 l., SQLite)
    └─ atlas_engine (684 l., embeddings)
    └─ skills SQLite (500 chars max)
```

Nouvelle architecture

```
Santana (monolithe modulaire, 17k LOC)
├─ Transport Layer      (Telegram handler → file de messages)
├─ Orchestrator         (react_loop + 27 outils)
│   ├── Provider Chain  (DeepSeek → OpenRouter free)
│   ├── Tool Registry   (27 outils + validation sorties)
│   ├── Cache SHA256    (TTL 5 min, ~/.cache/)
│   └─ Delegate         (sous-agents parallèles)
├─ Skills System        (fichiers .md versionnables)
│   ├── skill_view      (chargement dans contexte)
│   ├── skill_manage    (création/modification/suppression)
│   └─ skill_list       (inventaire)
├─ Memory System        (SQLite + profil injecté + Atlas)
├─ Logging              (JSON structuré → events.jsonl)
└─ Self-Correction      (checklist + avis asymétrique + Doctor)
```

3. Détail des correctifs – Phase 0 (P0)

P0.1 : fs_write

Fichier	Changement
tools/tools.py	Nouvelle fonction <code>tool_fs_write(path, content)</code> avec sandbox skills/ et workspace/
tools/tools.json	Nouveau schema pour fs_write
core/ react_loop.py	Ajout dans <code>_TOOL_PROGRESS</code> et catégorie meta

Impact : Santana peut créer et modifier ses propres fichiers (auto-amélioration).

P0.2 : Multi-provider LLM

Fichier	Changement
core/provider.py	Nouveau module – résolution de provider avec fallback inter-provider
deepseek_client.py	<code>complete()</code> et <code>complete_stream()</code> avec fallback vers core.provider
.env	Ajout <code>GROQ_API_KEY</code> , puis <code>OPENROUTER_API_KEY</code> + <code>OPENROUTER_MODEL</code>

Chaîne de fallback : DeepSeek V4 Flash → OpenRouter (Llama 3.3-70B, gratuit)

P0.3 : Skills en fichiers .md

Fichier	Changement
tools/skills_manager.py	Nouveau module – gestion de skills en fichiers .md
skills/rigueur-santana.md	Première skill installée (checklist + asymétrique)
tools/tools.py	Trois nouveaux outils : skill_view, skill_manage, skill_list
tools/tools.json	Schémas pour les 3 outils skills

P0.4 : delegate_task

Fichier	Changement
core/delegate.py	Nouveau module – délégation de tâches à des sous-agents
tools/tools.py	Nouvel outil delegate_task (sync wrapper async)
tools/tools.json	Schéma pour delegate_task

P0.5 : Validation des sorties d'outils

Fichier	Changement
core/react_loop.py	Nouvelle fonction _validate_tool_result() – filtre HTML, vide, taille

Vérifications : Contenu non vide, pas de page d'erreur HTML, limite 10 000 chars.

4. Détail des correctifs – Phase 1 (P1)

P1.1 : Healthcheck / Doctor

Fichier	Changement
scripts/santana-doctor.sh	Script de diagnostic (7 vérifications)
santana.py	Appel du doctor au démarrage, log des résultats

P1.2 : Zéro test rouge

Fichier	Changement
tests/ test_system_integrity.py	Test evaluator corrigé (vert maintenant, plus « échoue intentionnellement »)
scripts/pre-commit.sh	Hook pre-commit avec syntaxe + tests intégrité

P1.3 : Profil utilisateur injecté

Fichier	Changement
skills/profile-serge.md	Profil de Serge (identité, contexte tech, préférences, règles)
agent/orchestrator.py	Injection automatique du profil à chaque construction de prompt

P1.4 : Checklist de clôture

Fichier	Changement
agent/orchestrator.py	5 points obligatoires ajoutés dans le prompt système

P1.5 : SOUL.md – Charte d'identité

Fichier	Changement
soul/SOUL.md	Charte d'identité Santana (mission, valeurs, frontières, stack)

5. Détail des correctifs – Phase 2 (P2)

P2.1 : Avis asymétrique

Fichier	Changement
agent/orchestrator.py	Instruction asymétrique ajoutée au prompt système

P2.2 : Logs JSON structurés

Fichier	Changement
core/json_logger.py	Nouveau module – événements JSON avec correlation_id
core/react_loop.py	Instrumentation des appels outils

P2.3 : Cache SHA256

Fichier	Changement
core/cache.py	Nouveau module – cache par hash SHA256, TTL 5 min
core/react_loop.py	Cache lookup avant exécution outil, cache set après

P2.4 : Token counting amélioré

Fichier	Changement
agent/context.py	Estimation par mots/0.75 (vs len//4)
tests/test_context.py	Tests mis à jour pour le nouvel estimateur

6. Fichiers modifiés ou créés

Fichier	Action	Phase
core/provider.py	Créé	P0
core/delegate.py	Créé	P0
core/json_logger.py	Créé	P2
core/cache.py	Créé	P2
tools/skills_manager.py	Créé	P0
tools/youtube.py	Créé	Pré-P0
skills/rigueur-santana.md	Créé	P0
skills/profile-serge.md	Créé	P1

soul/SOUL.md	Créé	P1
scripts/santana-doctor.sh	Créé	P1
scripts/pre-commit.sh	Créé	P1
deepseek_client.py	Modifié	P0
tools/tools.py	Modifié	P0+P2
tools/tools.json	Modifié	P0+P2
core/react_loop.py	Modifié	P0+P2
agent/orchestrator.py	Modifié	P1+P2
agent/context.py	Modifié	P2
santana.py	Modifié	P1
.env	Modifié	P0

7. Résultats des tests

142	0	8.13s
Passés	Échecs	Durée

Suite de tests	Résultat
test_system_integrity.py	71/71
test_basics.py	22/22
test_utils.py	12/12

test_react_loop.py	14/14
test_context.py	10/10
test_code_exec.py	13/13

Doctor : 7/7 vérifications passées (process, DB, LLM, outils, skills, workspace, providers)

8. Métriques système

Métrique	Valeur
Fichiers Python	95
Lignes de code	17 013
Outils	27
Providers LLM	2 (DeepSeek + OpenRouter)
Skills .md	2
Mémoire process	476 MB
État service	Active (running)

9. Leçons apprises

- Copier n'est pas tricher.** Les 4 agents BTR ont identifié 40 choses à copier. Les 4/4 accords étaient les plus impactants.
- Architecture avant fonctionnalités.** Le Conseil a d'abord statué sur l'architecture (monolithe modulaire) avant d'ajouter des outils.

3. **Convergence multi-agent = priorisation fiable.** Les items 4/4 convergence étaient systématiquement les plus critiques.
4. **Le test rouge qui dure 5 jours est un aveu.** Un seul test qui échoue volontairement depuis le 19 juin a été découvert et corrigé.
5. **La chaîne de fallback gratuite existe.** OpenRouter donne accès à Llama 3.3-70B gratuitement — fallback robuste sans coût.

Document généré par Hermès (Black Intelligence) — 24 juin 2026
Source : github.com/NousResearch/hermes-agent • Licence MIT